ChatGPT

Tech Stack and Libraries

- **Backend language:** Use **Python** since it has mature libraries for text processing and ML (e.g. OpenAI API, Pandas, NumPy, sentence-transformers)【16†L844-L846】. A lightweight web framework like **FastAPI** or **Flask** can serve APIs.
- **Embedding models:** OpenAI's embeddings (e.g. `text-embedding-ada-002` via their API) or a local model like **SentenceTransformer (MiniLM)**. These convert text into vectors for similarity search【20†L156-L163】.
- **Vector store:** A dedicated vector database (e.g. **Chroma**, **Qdrant**, **Pinecone**) for efficient nearest-neighbor search, or a self-hosted FAISS/Annoy index. This stores chunk embeddings and metadata.
- **Data extraction:** For uploaded **PDFs/TXT** use PDF libraries (e.g. *PyMuPDF* or *PyPDF2* to extract text【18†L857-L860】). For website links, use **requests** + **BeautifulSoup** (bs4) to scrape and clean HTML【16†L921-L924】.
- **Frontend:** Any modern UI framework (e.g. React or Vue) for the web interface. The frontend shows the "Sources" list (uploaded files/URLs), deletion controls, and the Q&A chat. A file-picker and text input components are needed.
- **Database/Cache:** A database (SQL or NoSQL) to track user sessions and their current source list. Optionally use Redis for session/cache. Vector DBs often store metadata (source name, chunk ID) alongside embeddings.

Key libraries:

- *PyPDF2/PyMuPDF* for PDF text extraction【18†L857-L860】.
- *BeautifulSoup4* for HTML scraping【16†L921-L924】.
- *OpenAI SDK* or *sentence-transformers* for embeddings.
- *ChromaDB* (Python client) or *Qdrant*, etc., for vector search.
- *Whoosh* or *elasticlunr* (optional) if lexical/BM25 search is desired.
- *OpenAI GPT-4o/GPT-3.5* for answer generation.

System Architecture and Data Flow

We propose a **multi-component architecture** (see figure below) where user inputs and queries flow through modular services:

1. **User Interface (Frontend):** The UI lets users upload up to 5 sources (PDFs or URLs) and manage them (view/delete). Once a source is added, it is sent to the backend. A Q&A input box is shown but only enabled when at least one source is active (or if a "general question" mode is allowed).

2. **API Gateway / Backend:** A Python-based web server (e.g. FastAPI) exposes endpoints:

3. `/upload` to receive files or URLs.

4. `/delete` to remove a source.
5. `/list-sources` to list current sources.
6. `/ask` to submit a query.

7. Each endpoint validates input, checks user/session context, and orchestrates tasks.

8. **Ingestion Service:** When a new source arrives, the backend invokes the ingestion pipeline:

9. **Text Extraction:** For a PDF, use PyPDF2/PyMuPDF to extract raw text【18†L857-L860】. For a URL, use `requests` to fetch the HTML and BeautifulSoup to extract the article text【16†L921-L924】 (stripping scripts, navigation, ads).

10. **Preprocessing:** Clean the text (normalize whitespace, remove boilerplate, optional language filtering). Possibly auto-detect language to ensure compatibility.
11. **Chunking:** Split the text into manageable chunks (e.g. 500-word segments with some overlap). You can use fixed-length or sentence-boundary chunking. (Optionally use "smart chunking" that tries to split at paragraph/section boundaries.) If some chunks exceed token limits (e.g. 8K tokens for GPT-4o), further sub-split them.
12. **Embedding:** Call an embedding model on each chunk (via OpenAI API or a local model). This yields a vector per chunk.
13. **Storage:** Insert each chunk, its vector, and metadata (source ID, page/URL, chunk index) into the vector store. Also store the original text for retrieval.
14. **Vector Store and Metadata DB:** The vector store (Chroma, etc.) is the core search index. Each entry includes the chunk text, embedding, and references to the source. A relational DB or the same vector DB can hold metadata like source titles, upload time, user ID, etc.

15. **Query Handling:** When the user asks a question:

16. **Embedding Query:** The backend embeds the user's query with the same embedding model.

17. **Retrieval:** Perform a k-nearest-neighbor search in the vector database. For example, fetch the top K ≈ 5–10 chunks whose embeddings are most similar (cosine similarity) to the query vector【20†L156-L163】. (Optionally also run a keyword/BM25 search over text as a hybrid approach.)
18. **Ranking/Fusion:** You may combine results from embedding search and any keyword search. Techniques like *reciprocal rank fusion (RRF)* can merge these ranked lists. This yields a small set of relevant text chunks.

19. **Answer Generation:** Take the top retrieved chunks and prepend a system prompt. For example:

    ```
    You are an assistant that answers questions using ONLY the information
    in the provided sources. If the answer is not found in the sources, say
    "I don't have that information". Answer succinctly.
    Context:
    [Chunk1 from Source A...]
    [Chunk2 from Source B...]
    ...
    Question: <user query>
    Answer:
    ```

    The LLM (GPT-4o or GPT-3.5) is then invoked to generate the answer from this context.

20. **Fallback Handling:** As a safety, include a rule in the prompt: "Answer only from the provided context; do not use outside knowledge. If the context does not contain the answer, respond 'I

don't know'." This matches best practices: e.g. guiding models to "answer only from the provided context and not external facts" with a defined fallback【25†L1291-L1293】.

21. **Response to User:** The system returns the LLM's answer. Optionally, you can highlight or cite the source(s) it used for transparency (e.g. by showing which document/chunk contained the answer).

Data Flow Diagram (Textual)

```
User → Frontend (upload URL/PDF)
      → Backend API → Ingestion Pipeline
         → Text extraction → Chunking → Embedding → Vector Store

User → Frontend (ask query)
      → Backend API → Embedding → Vector Search → LLM Prompting
      → LLM Response → Backend → Frontend (display answer)
```

This architecture is microservices-friendly: each component (ingestion, retrieval, LLM) can be scaled or replaced independently.

Document Ingestion and Embedding Pipeline

1. Receiving sources: The backend limits each user/session to 5 active sources. Each source is either an uploaded file (PDF, TXT) or a URL. For uploads, store the file temporarily (or in S3). For URLs, store the link.

2. Text Extraction:

- **PDFs:** Use a library like *PyPDF2* or *PyMuPDF*. For example, PyPDF2's `PdfReader` can extract text from each page【18†L857-L860】. If PDFs contain images or scans, you might fall back on OCR (optional).
- **Text files:** Simply read the file as UTF-8 text.
- **Websites:** Fetch the HTML with `requests` (or `urllib`), then use *BeautifulSoup* to parse and extract relevant text (e.g. article body). Drop navigation menus, ads, etc. Regex or rules can help remove boilerplate.

3. Cleaning: Remove extra whitespace, fix encoding issues, filter out very short sections (like navigation crumbs). For multilingual docs, detect language to ensure embedding models handle it (most models support multiple languages).

4. Chunking: Split the cleaned text into chunks. A common approach is fixed-size windows (e.g. ~500 tokens, with 50-token overlap) so that each chunk fits the model's context. Alternatively, use a semantic chunker that splits at paragraph or heading boundaries. For long documents, this ensures we cover all content without exceeding token limits.

5. Embedding: Convert each chunk to a vector. For example, send the chunk text to the OpenAI Embeddings API (like `text-embedding-ada-002`) or a SentenceTransformers model locally. This

generates a high-dimensional embedding (e.g. 1536-D or 384-D) representing the chunk's meaning. Do this in batches for efficiency. Cache or store embeddings since chunks don't change after ingestion.

6. Storing: Insert each chunk's text and embedding into the vector database. Also record metadata: source ID (filename or URL), chunk index, text snippet, and any tags (e.g. "User Doc"). Some vector DBs allow JSON metadata fields. If using a simple vector store, keep a parallel SQL/NoSQL table for metadata.

7. Update Vector Store: Each time a source is added or removed, update the vector store accordingly. If a source is deleted, remove its chunks from the index. The system must maintain consistency between the user's active sources and the search index.

This ingestion pipeline is essentially a custom RAG feeder: it "ingests" user-provided knowledge, converting it into a searchable form【20†L156-L163】.

Retrieval, Ranking, and LLM Prompting

1. Query embedding & search: When a question arrives, first embed the query (same model as for chunks). Then search the vector store: find the **top-K** chunks most similar to the query (e.g. K=5). This uses cosine similarity or dot-product on the embedding vectors. For example, convert query and chunk vectors to NumPy arrays and find nearest neighbors (as in the OpenAI tutorial【6†L72-L80】).

2. Hybrid search (optional): To improve recall, you can also run a keyword search (BM25) on the raw text of all chunks (using a tool like Whoosh or ElasticSearch). Then combine (fuse) the semantic and lexical ranks, e.g. via **Reciprocal Rank Fusion (RRF)**, to form a final top-5 list. This helps catch cases where keywords matter but embeddings alone might miss.

3. Context assembly: Concatenate the selected chunks (or relevant excerpts) into the prompt. Add clear demarcation (e.g. `--- CONTEXT START ---`). Example system/prompt instructions:

```
You are a knowledgeable assistant. Use ONLY the information provided in the
context below to answer the question. If the answer is not in the context,
say "I don't have that information."

Context:
[Source A] "...text chunk 1..."
[Source B] "...text chunk 2..."
...
Question: <user's question>
Answer:
```

This follows recommended RAG prompting: "augment the user input by adding the relevant retrieved data in context"【20†L168-L170】. We explicitly instruct the model to avoid hallucinations and only use the given text. The Context-CoT guidelines even stress requiring the model to "answer only from the provided context" with a defined fallback【25†L1291-L1293】.

4. Answer generation: Call the chat model (e.g. GPT-4o or GPT-3.5-turbo). The model processes the combined context+question and generates a concise answer. Since we want short, user-satisfying

answers, we can prompt the model to be brief and to cite only from context. For efficiency, choose the smallest model that meets needs (e.g. GPT-3.5 for simple queries to reduce token cost, GPT-4o for complex ones).

5. Handling out-of-scope questions: If the user asks something unrelated (e.g. “Who won the World Cup?”) and the model has no data to answer from the provided sources, it should politely say so. By instructing a fallback like “If the context does not contain the answer, respond ‘I don’t know’”【25†L1291-L1293】, we ensure compliance. The system can also block the query field if no source is provided or if “general questions” mode is disabled, as requested.

6. Token efficiency: To minimize tokens per request, only send the few most relevant chunks (rather than all text). Also, consider summarizing very long chunks (or pruning them) if they exceed token budgets. In production, you might pre-generate summaries of large chunks to reduce prompt size, but that adds complexity. At minimum, ensure chunks are not overly large.

7. Returning the answer: The backend returns the LLM’s response to the frontend. You may also display which source(s) the answer came from (by highlighting or listing source names). This boosts transparency and trust.

In summary, the retrieval process uses vector similarity to fetch context, then a tailored LLM prompt to generate the answer, following best practices from RAG literature【20†L156-L163】【20†L168-L170】. The user experience is a chat-like interface that always respects the provided documents.

Deployment, Scaling, and Monitoring

- **Hosting:** Containerize the app (e.g. Docker) so you can deploy on any cloud or on-premise server. Use a platform like AWS (EKS/ECS), Azure, or GCP. The API server (FastAPI) can scale out behind a load balancer. The front-end is a static site (React) served via CDN or with a Node server.
- **State management:** If building for multiple users, implement authentication (OAuth or simple login) so each user’s sources are isolated. If it’s single-user, sessions can track up to 5 active sources. Store metadata (user ID, source list) in a database (Postgres or similar).
- **Vector store scaling:** For light usage, a local ChromaDB or in-memory FAISS may suffice. For production, a cloud vector DB (like Pinecone or Qdrant Cloud) can handle larger data and auto-scale. The ingestion pipeline can run on demand or via a job queue (e.g. Celery or AsyncIO workers).
- **LLM usage:** Use a managed API (OpenAI) to avoid running large models yourself. This offloads scaling and GPU management. Be mindful of rate limits; you may need request batching or per-user quotas.
- **Security:**
- Sanitize uploads to prevent malicious files. Only allow expected file types (e.g. PDF, TXT).
- Run the backend over HTTPS. Protect API keys (e.g. OpenAI keys) in secure storage.
- If scraping websites, respect robots.txt and be cautious of heavy scraping. Limit request size/timeouts.
- **Monitoring and Logging:**
- Log each user query and response (anonymized) for debugging and usage metrics.
- Track performance: LLM latency, embedding latency, search latency.
- Use tools like Prometheus/Grafana for metrics, and Sentry/Stackdriver for error monitoring.
- Monitor vector store health (disk use, query throughput).

- Set up alerts on high error rates or downtime.

- **Cost optimization:**

- Cache embeddings for identical chunks to avoid recomputing.
- Limit LLM token usage by setting appropriate truncation.
- Possibly throttle frequent queries.
- **Testing and CI/CD:** Write unit/integration tests for ingestion and retrieval. Automate deployment (e.g. GitHub Actions) to push updates to production safely.

This full-stack design – from document ingestion through LLM answer generation – creates a robust RAG Q&A system tailored to user-provided sources. It uses common production practices (containerization, modular services, vector search indexing) and enforces that answers come only from the user's data【25†L1291-L1293】【20†L156-L163】.
